

Pakka, explained

A guide to what this project is, how it decides things, and how every piece of it works. The first half uses no jargon. The second half goes all the way down.

Part one: what it is

Pakka checks a message before you act on it.

You give it a text, a WhatsApp forward, an email, anything. It answers three questions, in this order:

1. **Is this a scam?** A verdict, in words, not a percentage.
2. **Which part of it is the problem?** The exact phrases, quoted back to you and highlighted inside your own message.
3. **What do I do now?** Numbered steps, and a reply you can forward straight back to whoever sent it to you.

That third one is the part most tools skip, and it is the part that decides whether anyone is actually helped.

Why it exists

Most people who get a scam message already sense something is off. That is not where they get stuck. They get stuck on two things:

- They cannot point at **what exactly** is wrong.
- There is **nobody to ask** in the thirty seconds before they tap the link.

So they either pay, or they forward it to whoever in the family is considered good with phones, and wait.

The whole product is that second person, available immediately.

The half of the problem that is harder

Catching scams is the easy half. The hard half is **not** flagging ordinary messages.

A tool that shouts at everything gets ignored, and an ignored tool is worse than no tool, because the one time it is right nobody is listening. So Pakka is tested as hard on messages that must come back clean as on ones that must not:

- a genuine bank OTP (“Your OTP is 452891. Never share this with anyone.”)
- a real RBI periodic-KYC reminder, which tells you to visit a branch
- a delivery message that contains an OTP and tells you to share it with the agent
- a friend asking for your account number to send you trip money
- marketing full of urgency words: “last chance”, “today only”, “limited time”

All of those are in the test set, and all of them must score zero.

Part two: how it decides, in plain words

Three things look at your message. They are not equal, and the order matters.

1. Rules decide

Forty-seven rules, written as ordinary Python. Each one knows a single shape that scams take, carries a weight, and can point at the words that set it off.

A rule is not just a pattern. It also carries the **reason it exists**, and that reason is what the page shows you:

A QR code to receive money. Scanning a QR code can only send money out of your account. It can never bring money in. Anyone telling you to scan one to claim something is describing a thing that cannot happen.

The weights add up to a score, and the score picks a band:

score	verdict
8 or more	Almost certainly a scam
5 to 7	Likely a scam
2 to 4	Be careful
1	One thing to check
0	Nothing suspicious found

No model votes here. The same message always gives the same answer, and every answer can be read line by line.

2. A model widens the net

Rules have perfect precision on what they describe and **no opinion at all about anything else**. A scam phrased in words nobody wrote down scores zero.

So there is also a small statistical model, trained on 5,729 labelled messages. It has an opinion about everything and is wrong more often. Together they are better than either alone.

The model is deliberately a **linear** one, and that is the important design choice. A linear model's output is literally the sum of its per-feature contributions, so you can lay those contributions back on the text and show which characters it reacted to. It never has to be taken on trust.

3. Some checks are not about words at all

A pattern can look for the word “sbi”. It cannot tell you that the **a** in `amazon.in` is Cyrillic, or that `xn--80ak6aa92e.com` is displayed in the address bar as `apple.com`. Those are properties of the characters and the structure of the address, so a separate module answers them and reports the reason in words.

The boundary between them

This is the single most important rule in the whole project:

The model may raise concern. It may never clear a message the rules flagged.

If no rule fires but the model is confident, the page says so plainly: *“No rule fired, but this still reads like a scam.”* And the number in the dial switches from the rule score to the model’s reading, because a green **0** next to an amber warning reads as “safe” to anyone glancing at it, and the number is the part people glance at.

That bucket, where the model is worried and no rule can explain why, is also where the next rule comes from.

Part three: one message, start to finish

Take this, pasted into the box:

```
Dear customer your KYC has expired. Click http://sbi-verify-kyc.xyz
and share OTP with our executive to reactivate within 2 hours.
```

Step 1, normalise. The text is cleaned up into a form the rules can match against: accents dropped, digits standing in for letters mapped back, letters that have been spaced apart pulled together. Crucially, a map is kept from every character in the cleaned copy back to where it came from in the original, so highlights still land on what you actually pasted.

Step 2, run 46 patterns. Four fire:

```
+4 ASKS_FOR_SECRET "share OTP"
+3 KYC_PANIC       "KYC has expired"
+3 LOOKALIKE_DOMAIN "http://sbi-verify-kyc."
+1 URGENCY         "within 2 hours"
```

Step 3, the structural check. `urls.py` looks at the address itself and reports why it is wrong: the brand name `sbi` sits in front of a hyphen and a domain nobody at SBI owns.

Step 4, score and band. 11 points, which is “Almost certainly a scam”.

Step 5, the model. Independently scores it 0.98. It agrees, and it says which phrases it reacted to.

Step 6, advice. The rules that fired decide which steps apply. Because `ASKS_FOR_SECRET` is among them, the never-share-an-OTP line is added. Because the message impersonates a bank, the call-the-number-on-your-own-card line is added. Then the universal ones: report at `cybercrime.gov.in` or 1930, inside the first hour while money can still be frozen.

Step 7, the reply. Built only from the rules that actually fired, so it cannot overstate the verdict:

```
I checked this before replying — it uses a KYC or account-block scare and it asks
for your OTP. That is how this kind of scam works, so I am not paying or sharing
anything. Please do not send money either. (Checked with Pakka)
```

Step 8, the link. Only now does anything leave your device, and only if you press Share. The verdict is saved so it can be sent to whoever forwarded the message, and it deletes itself after thirty days.

Part four: the code, file by file

What ships to the server (`api/`, 10 files, no dependencies but boto3)

file	lines	what it does
<code>rules.py</code>	714	the 47 rules, the normaliser, the scorer
<code>handler.py</code>	262	one Lambda, every route
<code>hindi.py</code>	174	every line a person reads, in Hindi
<code>advice.py</code>	118	what to do now, and the reply to forward
<code>model.py</code>	116	the linear model, scored server side
<code>urls.py</code>	96	what a pattern cannot see about a web address
<code>chat.py</code>	100	the reply text, shared by every chat front door
<code>telegram.py</code>	96	the Telegram bot
<code>twilio.py</code>	85	WhatsApp through Twilio
<code>whatsapp.py</code>	106	WhatsApp through Meta's own API
<code>store.py</code>	76	DynamoDB, or a JSON file when there is no AWS
<code>features.py</code>	71	text into numbers, identically in two languages

What runs in the browser (`web/`)

`app.js` is the page. `rules.generated.js` and `model.generated.js` are **generated**, never edited. `model.js`, `urls.js` and `il8n.js` are the browser halves of things that also exist in Python. `scene.js` is the constellation. `sw.js` makes the app installable, which is what puts it in the Android share sheet.

What is only for development (`tools/`)

`train_model.py` trains and measures the model. `build_rules_js.py` generates the browser copy of the rules. `check_parity.py` proves the two copies agree. `eval.py` and `eval_set.py` are the labelled corpus. `audit_rules.py` checks the rulebook against somebody else's data. `devset.py`, `testset.py` and `finalset.py` are real scam messages collected from published sources. `try.py` checks a message from the command line.

Part five: the six ideas worth understanding

1. Normalisation, and why it needs an index map

Scam messages are written to get past filters. `K Y C` spaced out. `0TP` with a zero. `shäre` with an accent. `C-l-i-c-k` with hyphens. You could write a pattern for each trick, or you could undo the tricks once and keep the patterns simple. Pakka does the second.

`normalise()` does four things:

1. **Decomposes and drops accents.** `ä` becomes `a`.
2. **Maps digits that are standing in for letters** — `0` → `o`, `1` → `i`, `3` → `e` — but only where a letter sits beside the digit. So `0TP` becomes `oTP` and `acc0unt` becomes `account`, while a phone number and a rupee amount are left completely alone.
3. **Pulls apart-spaced letters together.** A run of three or more single letters separated by the *same* character collapses: `K Y C` → `KYC`, `C-l-i-c-k` → `Click`. The separator has to be consistent, otherwise `W-O-N a prize` would collapse into `WONa`.
4. **Replaces anything outside the basic plane with a placeholder**, so an emoji counts as one character rather than two.

The subtle part is the return value. It gives back **both** the cleaned text **and** a list mapping each cleaned character to its position in the original:

```
def normalise(text: str) -> tuple[str, list[int]]:
```

Without that map, a highlight computed on the cleaned copy would land in the wrong place in the message you pasted. With it, `Your K Y C has expired` highlights `K Y C` `has expire` — in the original spacing — even though the rule matched `KYC`.

The last step grows every span out to whole words, so you never see `KYC has expire` with a stranded `d`. That word test has to know that a Hindi vowel sign is a combining mark: `str.isalnum()` says `False` for it, which would stop a highlight mid-word and give you `ेड` where the word is `अपडेट`.

2. The rules engine

A rule is a frozen dataclass:

```
@dataclass(frozen=True)
class Rule:
    id: str          # LOOKALIKE_DOMAIN
    name: str         # "A web address dressed up as a real company"
    why: str          # the sentence the page shows you
    weight: int       # 1 to 4
    pattern: re.Pattern
```

`evaluate()` normalises once, runs every pattern against the normalised copy, maps the spans back, adds the weights, and picks a band. One rule, `LOOKALIKE_URL`, has a pattern that can never match: it is answered by `urls.py` instead, and appended last so that equal weights tie in a predictable order.

Weights are hand-assigned, 1 to 4, on one principle: **how often is this shape wrong when it appears in an ordinary message?** Asking for an OTP is a 4 because nothing legitimate does it. Urgency is a 1 because real messages are sometimes urgent.

3. Things a pattern cannot see

`urls.py` looks at the address rather than the sentence:

- **Mixed scripts in one word.** `amazon` written with a Cyrillic `а` renders identically to the real thing and resolves somewhere else. The check asks which Unicode scripts the letters of each label belong to, and a word using two is a word pretending to be another word.
- **Punycode.** `xn--80ak6aa92e.com` is what is stored; `apple.com` is what the address bar shows. Browsers give you the `xn--` form, which is exactly the form that hides the problem, so `web/urls.js` contains an RFC 3492 decoder written out by hand. It produces character-identical output to Python's IDNA decoder across the whole corpus, which the parity check verifies.
- **Digits inside a brand word.** `amaz0n-delivery.info`.
- **A bare IP address**, which is not a name anybody registered.

Each one returns a sentence, not a flag, because a finding nobody can explain is worth nothing.

4. The model

Features. The message is normalised, lowercased, and cut into every character sequence of length 3, 4 and 5. Character n-grams rather than words, because scam messages are full of misspellings, spacing tricks and Hinglish, and `kyc` inside `kycupdate` carries the same signal as the word alone.

Each n-gram is hashed with 32-bit FNV-1a into one of 16,384 buckets. Hashing instead of a dictionary means the feature space is fixed, the file is small, and a word never seen in training still lands somewhere sensible. Buckets are deduplicated and the vector is L2 normalised, so a long message does not simply outvote a short one.

Training. Logistic regression by stochastic gradient descent, written out in about thirty lines rather than imported, so the maths is auditable. Three details matter:

- **Positives are upweighted**, because the data is roughly eight to one against them.

- **The Indian corpus is upweighted twelvefold** on top of that, because it is 2% of the rows and 100% of the domain anyone cares about.
- **L1 regularisation by soft thresholding**, once per epoch. Plain magnitude pruning barely dented the model because SGD leaves almost every bucket slightly non-zero. L1 drives the useless ones to exactly zero, which took the shipped file from 245 KB to **12 KB**. That is the difference between shipping a model to a phone and not.

Calibration. The raw output of a logistic regression is a number between 0 and 1, but it is not a probability you can quote. A second one-dimensional logistic fit (Platt scaling) is applied on top, fitted **only on out-of-fold predictions**, never on anything the model trained on. That makes 0.8 mean roughly eight in ten:

predicted	actually fraud
0.0 – 0.2	0.20
0.2 – 0.4	0.32
0.4 – 0.6	0.45
0.6 – 0.8	0.68
0.8 – 1.0	0.94

That table is not decoration. It is why the page requires 0.75 before it will say “this reads like a scam”: at 0.6 the odds are only about seven in ten, and telling somebody their delivery notification is a scam on those odds is how a tool stops being believed.

Explainability. Each bucket’s weight is divided across the characters its n-gram came from, producing a heat value per character. Contiguous runs above a threshold are the phrases the model reacted to, and they are grown out to whole words before being shown. That is why it can say “*It reacted most to Your KYC, share, now within*” rather than asking you to trust a number.

Does it earn its place? The Indian corpus cannot answer that, because the rules were written against it and score 100% on it. The UCI set can, honestly, because no rule here has ever seen a 2012 British SMS. On 1,115 held-out UCI messages:

	precision	recall
rules alone	0.79	0.08
model alone	0.93	0.90

The rules are nearly blind out of domain. The model catches **120 spam messages no rule fires on**. That is the entire argument for it.

5. Generated code, and the invariant that keeps it honest

The rules are written once, in Python, and **generated** into JavaScript. The model is trained once, in Python, and **generated** into a JavaScript file of 789 numbers. Nothing is written twice by hand.

That gives the page a genuinely useful property: the check runs on the device. Nothing you paste leaves your phone unless you press Share. The whole engine is about 12 KB of arithmetic.

It also creates the obvious risk: two copies that drift. So `check_parity.py` runs 155 messages through Python and through Node and compares:

- the score, the band, and which rule ids fired
- the **reason text**, including the dynamically generated URL explanations
- the normalised text, character for character
- every highlighted quote
- the model's probability to six decimal places, and the phrases it reacted to

It is currently 155 out of 155, including Hindi in Devanagari. Three real bugs were caught only by this check, and all three were the same shape: an emoji is one character to Python and two to JavaScript, so `{0,50}` spanned different amounts of text; findings sorted differently when weights tied; and the two languages disagreed about whether a Devanagari vowel sign is part of a word.

A generated file is only trustworthy if something checks the generation.

6. Fusion, and who gets the last word

```
rules → verdict, band, quoted phrases, reasons      (decides)
model → a calibrated probability + its own phrases  (may raise, never clears)
urls  → structural findings, reported as sentences (a rule like any other)
```

Four combinations, and each says something different:

rules	model	what the page says
fired	high	"The model agrees with the rules independently"
fired	low	"The model is less sure. The rules above quote the exact words, so they are the ones to read."
nothing	high	" No rule fired, but this still reads like a scam. " Headline changes to <i>Nothing matched, but be careful</i> , dial shows the model's number
nothing	mid	"Nothing matched, and the model is not certain either. Around half of the messages it reads that way are fine."

Part six: where it runs, and what each AWS piece actually does

Everything is in **ap-south-1, Mumbai**, because the people using this are in India and their messages should not leave the country to be checked.

```
phone / browser
  |
  | the whole check already ran here, offline
  |
  v (only when you press Share, or a bot receives a message)
Lambda Function URL —> Lambda (Python 3.12) —> DynamoDB
    ^                                     |          (30-day TTL)
    |                                     v
Telegram / Twilio                      CloudWatch Logs
```

Six services. Here is what each one is, what it does here, and why it was chosen over the obvious alternative.

What “serverless” actually means here

There is no machine that belongs to this project. Nothing is running right now.

When a request arrives, AWS finds a spare slot on a machine it already has, unzips a 90 KB bundle of Python into it, runs one function, sends the answer back, and eventually throws the slot away. Between requests, nothing exists and nothing is billed.

That is either a brilliant fit or a terrible one depending on the shape of your traffic. For a tool people open **only when something looks wrong**, traffic is spiky and mostly zero. A server rented by the hour would sit idle almost all the time and still cost money every hour. That is the whole argument.

AWS Lambda: the code

What it is. A service that runs a single function in response to an event. You hand it a zip file and the name of a function inside it. AWS handles machines, scaling, patching and restarts.

What it does here. `api/handler.py` contains one function:

```
def lambda_handler(event, context=None):
```

`event` is a dictionary describing the HTTP request: method, path, headers, query string, body. Everything in this project comes in through that one function, which reads the path and decides what to do:

path	what happens
POST /scan	scan a message, save the verdict, return it with an id
GET /v/{id}	fetch a saved verdict, for whoever you sent the link to
POST /telegram	a Telegram message, answered in the same chat
POST /twilio	a WhatsApp message, answered in the HTTP response itself
GET /whatsapp	Meta's one-time webhook verification handshake
GET /	a health check that reports how many rules are loaded

Settings that matter, and what they cost you if wrong:

- **Runtime Python 3.12**, memory **256 MB**, timeout **10 seconds**. The defaults are 128 MB and **3 seconds**, and the 3 seconds is a trap: a hello world fits inside it, but a first DynamoDB call on a cold start does not. It surfaces as `Internal Server Error` with an empty body and no hint that time was the problem.
- **Handler string** must be `handler.lambda_handler` — the filename, then the function name. The console creates functions expecting `lambda_function.lambda_handler`, and uploading a zip does **not** update it, which produces a 502 with a completely empty body.

Cold starts, and the one trick worth knowing. The first request into a new slot pays for unzipping and importing. That phase is called **init**, and it has two useful properties: it gets more CPU than the request itself, and it is **not billed**. So anything expensive that can happen at import time should:

```
# api/store.py - at module level, not inside the request
_ddb = boto3.resource("dynamodb", config=Config(
    connect_timeout=3, read_timeout=3, retries={"max_attempts": 2}))
```

Building the DynamoDB client inside the request handler instead of at import was a real bug here: the first scan after an idle period ran out of time and came back without a share link, while every scan after it worked. Moving one line up a scope fixed it.

What it costs. One million requests a month are always free, plus 400,000 GB-seconds of compute. At 256 MB and ~80 ms a scan, this project would need roughly twenty million scans a month to leave the free tier.

Lambda Function URLs: the front door

What it is. A permanent HTTPS address attached directly to a Lambda function. You get something like `https://<id>.lambda-url.ap-south-1.on.aws/`, and anything posted to it invokes the function.

What it does here. It is the entire API. The page calls it to mint a share link, and Telegram and Twilio call it with incoming messages.

Why not API Gateway, which is what every tutorial reaches for. API Gateway is a full HTTP router with stages, deployments, request validation, throttling and usage plans. This project needs exactly one public endpoint with CORS. Function URLs carry **no additional charge**, while the API Gateway free tier expires after twelve months — so a project meant to stay free forever would start costing money in year two for a feature it never used.

Two things that will bite you:

- **The default auth is `AWS_IAM`.** You create a Function URL, whose entire purpose is to make a function publicly reachable, and the default makes it unreachable. Calling it returns `{"Message": "Forbidden"}` with no mention of authentication, no mention of the setting and no link. It has to be set to `NONE` for a public endpoint.
- **CORS stacks.** If you configure CORS on the Function URL, AWS adds the headers itself. If your code also adds them, every response goes out with `Access-Control-Allow-Origin: *, *`, which every browser rejects. The evil part is how it fails: `curl` is perfectly happy and only the actual web page breaks. The code here only adds CORS headers when it detects it is *not* inside Lambda:

```
IN_LAMBDA = bool(os.environ.get("AWS_LAMBDA_FUNCTION_NAME"))
CORS = {} if IN_LAMBDA else {"Access-Control-Allow-Origin": "*", ...}
```

Amazon DynamoDB: the saved verdicts

What it is. A managed key-value and document database. You do not run a server, you do not write SQL, and you do not manage indexes unless you want them. You give each item a **partition key**, and you fetch items by that key.

What it does here. One table, `pakka-scans`, partition key `id`. A scan is written as a single item: the message, the score, the band, the findings, the advice, and an expiry timestamp. `GET /v/{id}` reads one item by key. There are no queries, no scans, no secondary indexes — the access pattern is literally “give me this one id”, which is exactly what DynamoDB is best at.

Provisioned at 1 read unit and 1 write unit. DynamoDB has two billing modes. On-demand charges per request and needs no thinking. Provisioned reserves capacity, and the always-free tier includes **25 read units and 25 write units forever**. Setting 1 and 1 means the cost is not just low, it is *certainly* zero, which matters more than optimal for a free tool. One write unit is one write per second of an item up to 1 KB — plenty for a tool nobody is using at scale yet, and if it were exceeded the write fails and the code already handles that by returning the verdict without a link.

Time to live, which is the best thing in this stack. Under Settings → Time to live you name one attribute. Here it is `expires_at`. Write a Unix timestamp into it, and DynamoDB deletes the item after that time, for free, in the background.

```
item["expires_at"] = int(time.time()) + 30 * 24 * 3600
```

Why this matters beyond convenience: this table holds **other people's private messages**. A saved scan is evidence for the person who nearly paid, not an archive anybody has business keeping. Data that expires by default is the right shape, and TTL turns that from a cleanup job you would write, schedule, monitor and eventually get wrong into one attribute name in a settings panel.

The gotcha. boto3 returns DynamoDB numbers as `Decimal`, which `json.dumps` refuses to serialise. So the very first thing anyone does with a stored item — hand it back as JSON — throws a `TypeError` from inside the standard library, nowhere near the DynamoDB call. The fix:

```
def _plain(value):
    if isinstance(value, decimal.Decimal):
        return int(value) if value == value.to_integral_value() else
            float(value)
    raise TypeError(...)

json.dumps(body, default=_plain)
```

AWS Amplify Hosting: the website

What it is. Static site hosting wired to a git repository, with a CDN in front and HTTPS included. Push to a branch and it redeploys.

What it does here. It serves `web/` at `https://main.dlnv1rv96k8kj1.amplifyapp.com/`. The configuration is one file in the repo:

```
# amplify.yml
frontend:
  phases:
    build:
      commands: []          # nothing to compile
  artifacts:
    baseDirectory: web
```

Why the empty build step is deliberate. There is no bundler, no framework, no transpiler. The front end is HTML, CSS and JavaScript that browsers already understand. That means build minutes stay at zero, there is nothing to keep up to date, and **nothing that can fail at deploy time**. The generated files (`rules.generated.js`, `model.generated.js`) are generated on a laptop and committed, so what is in git is exactly what ships.

Amazon CloudWatch Logs: how anything got fixed

What it is. Wherever Lambda's output goes. Every `print()` and every uncaught traceback lands in a **log group** named `/aws/lambda/<function>`, split into **log streams**, one per execution environment.

What it does here. It is the only reason this project works. Every problem described in this part — the 403, the empty 502, the timeout, the doubled CORS header — named itself in a traceback. The moment I stopped guessing and started reading the log group, each one took a couple of minutes instead of twenty.

The handler is built to cooperate with it. An uncaught exception in Lambda becomes a bare 502 with an empty body, which tells the caller nothing and the developer less. So it is caught, logged, and answered in the same JSON shape as everything else:

```
try:
    return _route(event)
except Exception as exc:
    traceback.print_exc()                                # -> CloudWatch
    return _reply(500, {"error": "Something broke handling that.",
                       "detail": f"{type(exc).__name__}: {exc}"[:300]})
```

That `detail` field is the difference between “it's broken” and knowing which line broke, without opening the console at all.

AWS IAM: permissions

What it is. Identity and Access Management — who is allowed to do what.

What it does here. The Lambda has an **execution role**, which is the identity the function assumes while it runs. That role grants exactly two kinds of thing: write to its own CloudWatch log group, and read/write **one** DynamoDB table. It cannot touch any other table, cannot create resources, cannot read secrets. If the function were ever compromised, the blast radius is one table of expiring scans.

This is the principle worth stating out loud: **a function should hold the smallest set of permissions that lets it do its job**, because permissions are the only thing standing between a bug and a breach.

AWS SAM: the whole stack as one file

What it is. The Serverless Application Model — an extension of CloudFormation, AWS's infrastructure-as-code service. You describe what you want in YAML and AWS builds it.

What it does here. `template.yaml` describes the function, the table, the TTL attribute, the Function URL and its CORS configuration, and the parameters for the chat integrations. That matters for two reasons. Anyone can deploy their own copy of this

from that one file, and any change to the infrastructure is reviewable in a pull request instead of being a thing somebody once clicked in a console and cannot remember.

What a single request actually costs

At a thousand scans a month, all of it is inside the free tier — and most of those thousand never touch AWS at all, because the check runs on the device and the cloud is only involved when somebody presses Share.

At a hundred thousand, Lambda invocations are still free, Amplify is still serving static files from a CDN, CloudWatch is inside its 5 GB, and the only line that starts costing anything is the DynamoDB writes.

And the part that is not AWS at all

The same handler runs with **no AWS account whatsoever**. `api/store.py` falls back to a local JSON file when `PAKKA_TABLE` is unset, and `python3 api/local_server.py` wraps the identical Lambda handler in a standard-library HTTP server:

```
result = lambda_handler({
    "httpMethod": method,
    "path": parsed.path,
    "queryStringParameters": {...},
    "headers": {...},
    "body": body,
})
```

That file exists to guarantee that what runs on a laptop is the code that runs in production rather than a sibling of it — and it earned its keep: it was once passing the path with the query string still attached and never building `queryStringParameters`, so a route matched on Lambda and missed locally. That is precisely the drift it exists to prevent, and finding it was the point.

Part seven: the four front doors

The scam arrives in a chat. Copying it out, opening a browser and pasting it is four things to do while somebody is rushing you, which is the moment you are least able to do them. So the answer comes back where the message already is.

door	how it works
The page	offline-first; the cloud is only for minting a shareable link
Telegram	<code>@pakka_check_bot</code> . Telegram has no request signature, so it echoes back a secret you registered with the webhook, on every call
WhatsApp via Twilio	the reply is returned in the HTTP response body as TwiML, so there is no outbound API call and no access token to store. Requests are verified with an HMAC-SHA1 signature over the URL plus every parameter in sorted order
Android share sheet	the page declares a Web Share Target, so once installed it appears in the share sheet of every app. Long-press the message in WhatsApp, Share, Pakka. The share arrives as a query string, which is then wiped from the address bar so a stranger's message is not left in browser history

There is a fifth, Meta's own WhatsApp Cloud API, written and tested and behind unset environment variables, because Meta requires a business account and an app review.

All of them share `chat.py`, which writes the reply. They differ only in how a message arrives and how one is sent. Every one of them returns 200 as soon as the signature passes, because both Telegram and Twilio retry anything else, and a retry would scan the message again and answer twice.

Part eight: Hindi

The engine read Devanagari and Hinglish for a while before the interface did, which is a strange thing for a tool built for people who mostly do not read English comfortably.

`hindi.py` holds every line a person reads: the five bands, all 47 rule names, all 47 reasons, the advice clauses, the numbered actions, the forwardable reply and the interface labels. It is kept beside the rules rather than inside them, so a rule stays one idea in one place and a translation is a lookup. Anything missing falls back to English, which is the right failure: an untranslated line is readable, a blank one is not.

Two details decide whether this reads as translated or as written:

- **Devanagari does not take letter-spacing.** The mono labels use wide tracking, which pulls conjuncts apart and makes Hindi look broken. It is disabled under `html[lang="hi"]`.
 - **The bots detect the script and answer in it.** Somebody who forwards a Hindi scam is not helped by an English explanation of it, and asking them to pick a language first is one more step at the worst possible moment.
-

Part nine: how well it actually works

This is the part to be careful about, because it is the part where it is easiest to fool yourself.

The corpus I wrote

155 labelled messages, 93 fraud across 22 families and 62 legitimate ones chosen to be hard. **93 of 93 caught, 0 of 62 false positives.**

That number flatters the rules, because the same person wrote both. The README says so next to it.

Somebody else's data

`audit_rules.py` runs the rules over an external Indian SMS corpus from Hugging Face. That dataset turns out to be **templated rather than collected**: its 5,951 spam rows share 41 distinct skeletons, 0.7% unique. Useless for training. Genuinely useful as an **inventory of shapes**, counted once each so a template repeated 353 times does not vote 353 times.

First run covered **29 of 41**. The twelve misses became three new rules and several widened patterns. Now **41 of 41**.

The audit also prints how often each rule fires across every message available, which is how two rules were found to be untested rather than useless, and got test cases instead of deletion.

Real messages, measured three times

Three sets of real scam messages, quoted from published sources including the government's own fact-check unit. Each was measured **once while frozen**, then deliberately spent fixing what it exposed, and a fresh set collected from different sources to find out whether the fixes generalised or only memorised.

round	messages	rules reach "be careful", first run	what it exposed
one	25	52%	fake transaction alerts, reward-point expiry, digital arrest, demands for secrecy, service suspension, failed-delivery address updates
two	20	75%	traffic challans, bills that ask to be "updated", mixed Latin and Devanagari
three	10	80%	government scheme names used as bait

52%, then 75%, then 80%, against 100% on the corpus I wrote.

That gap is the most useful thing this project measured about itself. It is also the clearest argument for the model, which catches 24 of those 25 where the rules caught 13.

If somebody asks you for one number, give them that one, and the reason it is lower than the other one.

Part ten: what is weak

Say these before anyone finds them.

1. **Every corpus is self-written or published-example.** All three holdouts are now spent. The next honest number needs real forwards from real phones, which is the single highest-value thing anyone could hand this project.
 2. **Published examples skew** towards campaigns big enough to be written about.
 3. **Ten-message sets cannot resolve a percentage** better than about ten points.
 4. **47 rules is 47 shapes.** A genuinely new scam is invisible to all of them, and the model is the only thing standing behind them.
 5. **The model is small and its training data is mostly foreign.** UCI is British SMS spam from 2012. It teaches “free”, “win”, “claim” and nothing about UPI, APK sideloading or AnyDesk.
 6. **The Twilio sandbox only answers numbers that sent the join code**, and the join expires after 72 hours of inactivity.
 7. **The share target is Android and Chrome.** iOS does not implement it.
-

Part eleven: the commands

```
# check one message without opening anything
python3 tools/try.py "Your KYC has expired, share OTP now"

# the unit tests: rules, and each front door end to end with the network
  stubbed
python3 api/test_rules.py
python3 api/test_telegram.py
python3 api/test_twilio.py
python3 api/test_whatsapp.py

# the labelled corpus, and every miss and false positive
python3 tools/eval.py -v

# does the rulebook cover shapes somebody else collected?
python3 tools/audit_rules.py

# real messages, measured
python3 tools/run_holdout.py           # the last set
python3 tools/run_holdout.py dev       # round one
python3 tools/run_holdout.py test      # round two

# train, cross-validate, calibrate and export the model
python3 tools/train_model.py

# regenerate the browser copies, then prove they agree with Python
python3 tools/build_rules_js.py
python3 tools/check_parity.py

# run the whole thing locally with no AWS account
python3 api/local_server.py
```

Part twelve: questions you will be asked

“Why not just use an LLM?” Three reasons, in order. A verdict has to be the same every time, and a model that is asked the same question twice may answer differently. A verdict has to survive being explained to the person who nearly paid, and “the model said so” does not. And it has to run on their phone with the network off, because the message is already on that phone and sending it somewhere is the thing they are afraid of. There *is* a model in here; it just does not get the last word.

“Isn’t this just regex?” The rules are, and that is their strength and their ceiling. Out of domain they catch 8% of real spam. That is exactly why there is also a calibrated linear model that catches 90%, and why the two are fused with the model explicitly subordinate. The honest answer is: the rules give you reasons, the model gives you reach, and the project measures both separately so you can see which is carrying which case.

“How do you know it works?” I measured it three times on messages I did not write, before using any of them to change anything, and the first number was 52%. It is now 80% on the last set. Everything is in the repository and re-runnable.

“What would you do next?” Get forty real forwards and build a holdout that is actually frozen. Everything else is a smaller gain than that one.

“What broke?” The most interesting one: a rule for “asks you to keep it from your family” fired on “*Never share this with anyone*” in a genuine bank OTP message. Safety advice read as a scam signal, the exact inverse of the intent. The fix was recognising that the tell is being told not to **talk to** someone, not being told to keep a code to yourself. It is a good example of why the false-positive half of the test set matters as much as the other half.